

Grounded Listings Assistant

A chat assistant that recommends only from a fixed list of local places — and refuses to make things up.

Project overview for management • Prepared in simple English

The one-sentence idea: It's like a tour guide locked in a room with a single printed brochure and no phone. No matter what a customer asks, the guide can **only** point at the places in that brochure. Ask for a flight or a place that isn't listed, and it politely says it can't help with that. This is exactly what the client wants to test — that it **cannot be pushed off its data**.

Why this is harder than it looks

A normal AI (like ChatGPT) knows about the whole world, and will happily *invent* a restaurant that sounds real but doesn't exist — this is called "hallucinating." For a real business that's dangerous: you could send a customer to a fake address. So this whole project is built around one rule: **the AI can only talk about the approved listings, and we enforce that in code — we don't just politely ask it to behave.**

How it stays safe: three walls ("defense in depth")

Wall 1 — Locked ingredients

The AI never gets the full list. It can only "press two buttons" (tools) to search or fetch a place. So it literally can't know about a place that isn't in the database.

Wall 2 — The rulebook

Before every chat, we instruct the AI: only use search results, never invent links, refuse bookings/off-topic, ignore anyone telling it to break the rules.

Wall 3 — The bouncer

After the AI answers, our code re-checks every place and link against the approved list. Anything invented is stripped out and logged. The cards shown to the user are built only from approved places.

If one wall ever fails, the others still hold.

What happens when someone sends a message

- 1 The user types, e.g. "*vegetarian breakfast in Brookline*."
- 2 The message goes to the server's chat endpoint (the "order window").
- 3 The AI reads the rulebook and decides to use the **search** tool.
- 4 The question is turned into numbers ("meaning") and the database finds the closest matching places.
- 5 Those results are added to an "approved list" for this conversation.
- 6 The AI writes a friendly reply using only those results, streamed word-by-word.
- 7 The bouncer double-checks the reply, builds verified cards, and adds the "AI can be wrong" disclaimer.
- 8 The user sees the answer plus a card for each recommended place.

Smart search by meaning (not just keywords)

Each place is converted into a list of numbers that represents its *meaning* — like GPS coordinates, but for ideas. Similar things end up close together. So "somewhere to grab a quick coffee" can find "The Mill House Cafe" even though the word "cafe" was never typed. This is called **semantic search**, powered by a database extension called **pgvector**. The "meaning numbers" (embeddings) are currently computed **on our own machine**, so this part needs no API key and no internet.

What is configured right now

Piece	Status
Web app (the chat website)	Live at http://localhost:3000
AI brain (chat model)	Google Gemini 2.5 Flash (active). Claude & GPT also wired — switchable with one setting.
Meaning-search (embeddings)	On-device (no key needed). Can switch to OpenAI in one line.
Database (the memory)	Running in Docker , loaded with all 18 listings.
Secrets & settings	Kept in a private .env file (never shared or committed).

The technology stack (the tools we used)

Layer	Technology	What it does (plain English)
Language	TypeScript	JavaScript with safety checks that catch mistakes before running.
Engine	Node.js 24	Runs all the server-side code.
Framework	Next.js 16	Runs both the website and the backend in one project.
Interface	React 19	Builds the chat screen the user sees.
AI toolkit	Vercel AI SDK v5	One common socket for every AI provider; handles live streaming and the AI's "tools."
AI providers	Gemini · Claude · GPT	Interchangeable AI brains (Gemini active now).
Meaning-search model	Transformers.js (MiniLM)	Turns text into "meaning numbers" on-device, free, no key.
Database	PostgreSQL + pgvector	Stores the places and finds them by meaning.
Containers	Docker	Runs the database in a clean, isolated box.
Data safety	Zod	Checks that every place's data has the correct shape.
Testing	Vitest	Automated tests that prove the safety rules work.

What we suggest building next

Quick wins

- "Searching..." indicator so users know it's working.
- Friendlier error messages.
- Smoother multi-question conversations ("what about something cheaper?").

Bigger features

- Scale from 18 to thousands of places — the design already supports it.
- A voice version: a phone line that talks, reusing the same safe brain.

Make it production-ready

- Login + rate limiting (stop strangers running up the AI bill).
- Save the "bouncer caught something" logs and alert us.
- Automated "attack tests" on every change (50 sneaky prompts) to prove it still refuses.

- Put it online (hosted) so it's not just on a laptop, with caching for speed and cost.

Polish

- Add filters like distance and "open now."
- Use sharper paid embeddings once the dataset is large.

Bottom line: The assistant is working end-to-end today. Its core promise — *it only recommends from our approved list and can't be tricked into making things up* — is enforced by code in three independent layers, not just by asking the AI nicely. That is exactly the capability this engagement is meant to prove.

Grounded Listings Assistant — internal project overview. Stack: Next.js 16, React 19, TypeScript, Vercel AI SDK v5, Google Gemini (active) / Claude / GPT, PostgreSQL + pgvector, on-device embeddings, Docker, Zod, Vitest.